



장준환

데이터로 문제를 해결하는 엔지니어



010-7356-9008



neon9008@gmail.com



[GitHub](#)



[Dev-Blog](#)

학력/교육

건국대학교 스마트운행체공학과 졸업 (4.2/4.5)	2017.03 - 2024.02
현대오토에버 모빌리티 SW 스쿨 클라우드 2기 수료	2025.04 - 2025.11
삼성청년SW·AI아카데미 (SSAFY) 15기	2026.01 -

경력

대중음악웹진 IZM 편집장	2019.03 - 2024.09
CJ ENM 아티스트 사업부 데이터 계약직	2024.03 - 2024.11

자격증

SQLD (SQL 개발자)	2024.11.17
네트워크관리사 2급	2025.09.30
정보처리기사	2026.06.12

어학

TOEIC / 770	2025.02.23
OPic / IM1	2025.03.13

수상

2019년 2학기 건국대학교 Dean's List 수상	
삼성청년SW·AI아카데미 1학기 성적우수상	2026.06.26

About Me

2024.01



IZM

PM, 데이터 설계
웹진 현대화

2024.03



CJ ENM

데이터 계약직
수집과 지표 분석

2025.04



현대오토에버

모빌리티 SW 스쿨
클라우드 2기

2025.08



digem

1인 풀스택
ETL 아카이브

2025.10



NEVES

팀 프로젝트
영상 스트리밍

○ 프로젝트 ● 경력과 교육

2019년부터 음악 웹진 IZM의 에디터와 편집장으로 일했습니다. 2024년에 20년 된 사이트가 해킹 피해를 입어 복구부터 하게 됐습니다. **인프라와 보안을 찾아보면서 사이트 이전 계획을 세웠고**, 요구를 말로 넘기면 해석이 갈려 PPT로 화면의 경우의 수를 하나씩 만들고 학부에서 배운 지식으로 ERD를 그려 개발사에 전달했습니다. **그 과정에서 문제 대부분이 노후화된 인프라와 데이터 구조에서 나온다는 것을 알게 됐습니다.**

CJ ENM 아티스트 사업부에서 데이터 계약직으로 **지표를 수집하고 분석해** 상부 회의 자료로 쓰일 보고서를 만들었습니다. 다만 **자동화를 몰라서** 수집 스크립트를 매번 손으로 실행했고, 엑셀로 저장된 결과를 다시 시트로 옮겨 정리했습니다. 이 분야를 더 자세히 배우고 싶어 클라우드 교육 과정에 들어갔습니다. **만들고 싶었던 데이터 ETL 파이프라인 구조를 공부했고, IZM 사이트를 이전할 때 썼던 AWS와 웹 서비스가 돌아가는 방식도 그때 제대로 배웠습니다.**

과정 중 팀 프로젝트로 만든 영상 스트리밍 서비스에서는 개발 경험이 없었기에 로그인과 메일 발송을 맡았고, 인프라와 스트리밍은 전공자 친구들이 담당했습니다. **대신 옆에서 보고 물으며 구조를 파악하는 데 집중했고, 더 배우고자 SSAFY로 전직했습니다.**

2026.01



SSAFY

삼성청년SW-AI아카데미
15기

2026.05



beautalk

팀장, 백엔드와 AI
추천 챗봇

2026.07



이음길

백엔드
실시간 협업

2026.08



Finch

팀장, 인프라
모의투자

1년이 지난 지금은 범위가 달라졌습니다. 화장품 추천 챗봇에서는 **크롤링부터 데이터 정제, 추천 엔진, 배포까지 백엔드 전체를** 맡았고, 음악 칼럼을 모아 번역하는 서비스는 혼자 만들어 1년째 매일 돌리고 있습니다. **그때 눈으로만 익혔던 인프라도 지금은 직접 맡고 있습니다.** 모의투자 서비스에서 nginx와 백엔드, AI 서버, 데이터베이스를 컨테이너로 묶어 올리고, **Prometheus로 지표를, Loki와 Alloy로 컨테이너 로그를 모아 Grafana 대시보드에서 함께 보는 관측 스택을** 구성했습니다. 백업 복원 절차도 직접 리허설해 실측값과 함께 기록해 두었습니다.

만드는 것보다 왜 만들어야 하는지, 돌아가는 데서 그치는 것이 아닌 어떻게 더 잘돌아가게 하는지 관심이 많습니다. 비용 때문에 아키텍처를 세 번 바꾼 것도, 실패를 유형으로 나눠 저장하기 시작한 것도 그 과정에서 나왔습니다.

다루는 기술



Python



Django



FastAPI



PostgreSQL



Redis



Airflow



Docker



Kubernetes



AWS



GitHub
Actions



Jenkins



Prometheus



Grafana



Git



Jira

1 IZM : 20년 레거시 음악 웹진 현대화

과거 사이트

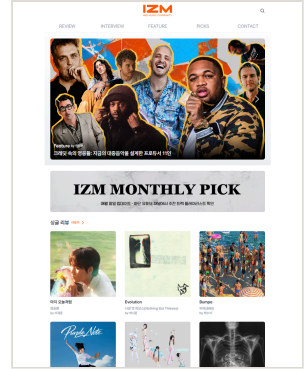
현재 사이트

2024.01 ~ 05

편집장으로 활동할 시절, 2001년 개설된 웹진(Classic ASP 기반)이 해킹 피해를 입어, 사이트를 복구하고 리뉴얼하면서 인프라와 데이터 구조까지 다시 설계한 프로젝트입니다. PM으로서 외부 개발사와의 미팅을 주도하고, 매주 스크럼 회의를 통해 요구사항을 전달하며 일정과 진행 상황을 관리했습니다. 요구를 말로 전달하면 해석이 갈리기에 PPT로 레이아웃의 경우의 수를 하나씩 만들었고, 학부 데이터베이스 수업에서 배운 지식으로 ERD를 그려 개선 스키마가 담긴 제안서를 작성해 전달했습니다.



기존 사이트



개편 사이트

1 콘텐츠와 아티스트의 연결 구조 개선

기존에는 콘텐츠 하나에 아티스트를 한 명만 연결할 수 있어, 협업곡과 컴필레이션을 등록하려면 우회책으로 아티스트 이름을 이어붙인 레코드를 새로 생성해야 했습니다.

- 콘텐츠와 아티스트의 직접 참여 관계는 유지하고, **관련 아티스트**를 위한 **별도 N:M 관계**를 추가했습니다.
- 관련 아티스트 태그를 기반으로, 아티스트 페이지에서 관련된 아티스트, 언급된 콘텐츠까지 탐색할 수 있도록 개선했습니다.

아티스트
사이먼 도미닉(Simon Dominic)
염따, 딥플로우, 팔로알토, 더콰이엇, 사이먼 도미닉
사이먼 도미닉 , 로코(Simon Dominic, Loco)
사이먼 도미닉 , 로코, 우원재, 쿠기

개선 전: 같은 아티스트가 참여 조합마다 다른 레코드



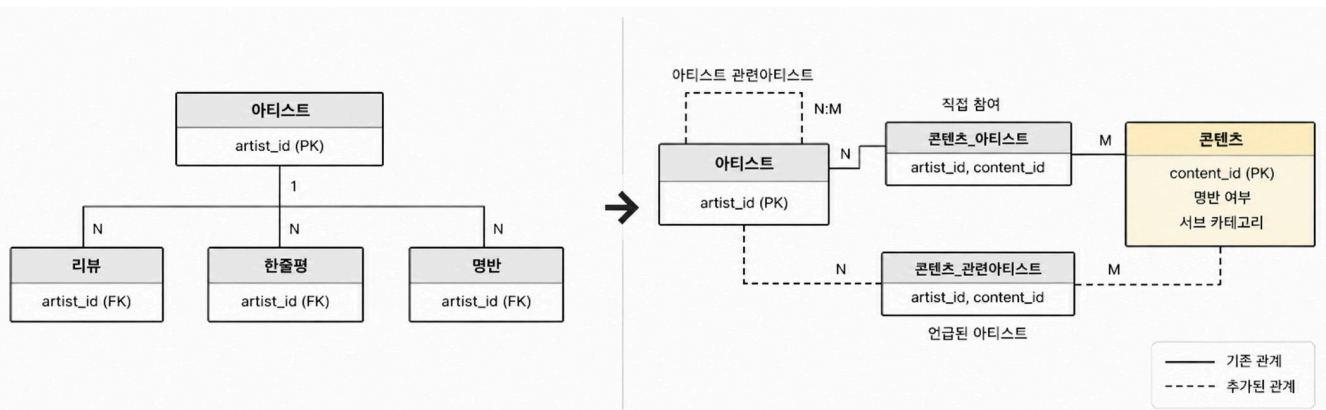
아마두
더 콰이엇(The Quiett), 팔로알토(Paloalto), 사이먼 도미닉(Simon Dominic), 딥플로우(Deepflow), 염따
Feat. 허클베리 피, 넉살, 우원재, 김효은

개선 후: 참여자를 각각 연결하고 피쳐링까지 구분

2 중복 등록을 줄이는 운영 모델 구축

기존에는 '리뷰', '한줄평', '명반'이 '아티스트'라는 테이블을 참조하는 별도 테이블로 존재했고, 같은 앨범도 리뷰용, 한줄평용, 명반용을 따로 등록해야 하는 불편함이 있었습니다.

- 모든 글은 '**콘텐츠**' 테이블로 통일하고 명반 선정 여부와 서브 카테고리를 **속성으로 관리**하도록 통합했습니다.
- 중복 등록을 방지하고, 하나의 콘텐츠를 목적에 따라 재사용할 수 있도록 개선했습니다.



2 digem : 글로벌 음악 소식 자동 수집 가공 아카이브

Github

사이트 이동

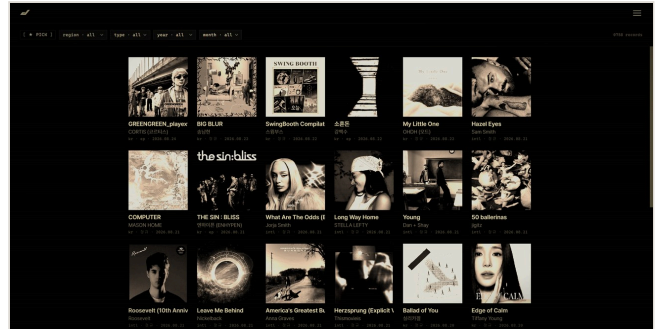
PW: digem123!

1인 / 풀스택 / 2025.08 ~ 진행 중

해외 음악 매체의 칼럼과 국내외 앨범 정보를 자동 수집하고, AI 번역을 거쳐 한국어 아카이브로 제공하는 **개인 ETL 서비스**입니다. 1년 넘게 **목적에 맞춰 아키텍처를 세 번 바꾸며**하고, 기획부터 데이터 수집, 자동화, ETL 파이프라인, 데이터 모델, 웹 서비스, 운영 환경까지 **전 과정을 구현**했습니다.

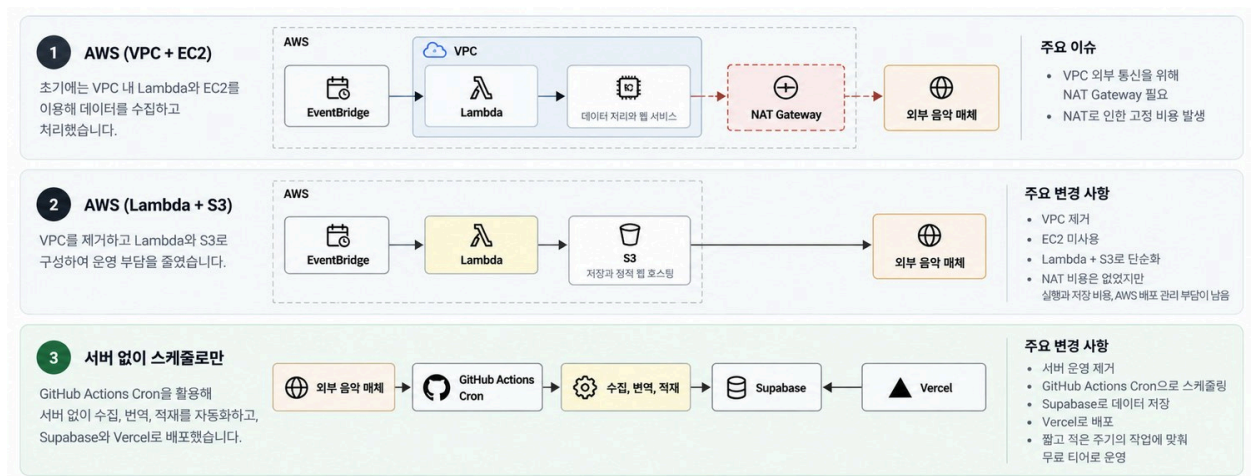


칼럼 페이지



앨범 페이지

아키텍처 전환



파이프라인 구성

1 수동 크롤러를 자동 ETL 파이프라인으로 전환

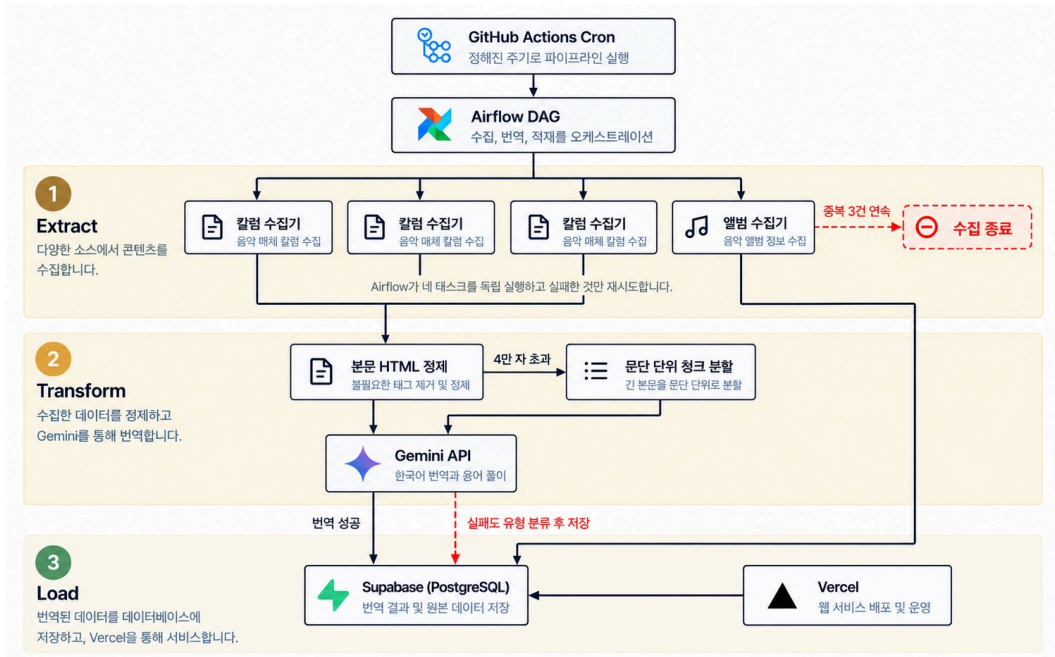
이전 직장에서 만든 수집 스크립트가 출발점이었습니다. 수동 실행이라 실행 주체가 없으면 신규 데이터 수집이 중단됐고, 파일 안에서 순차로 하나씩 실행돼 스크래퍼 하나가 실패하면 처음부터 다시 돌려야 했습니다.

- **GitHub Actions Cron**으로 수집, 번역, 적재 과정을 하루 한 번 자동 실행했습니다.
- Airflow DAG로 스크래퍼를 독립 태스크로 분리해 실행 순서와 실패한 것만 재시도하도록 관리했습니다.
- 매체마다 RSS 제공 여부, HTML 구조, 페이지네이션이 달라, 부모 클래스에 **수집, 파싱, 정제, 저장의 공통 흐름**을 두고 매체별 차이만 개별 구현했습니다.

2 불필요한 수집과 번역 API 호출 차단

앨범 수집기가 매 실행마다 최신 목록 전체를 순회했고, 번역에 실패한 기사를 건너뛰면 다음 실행 때 같은 기사를 다시 수집해 API 호출이 반복됐습니다.

- 중복 데이터가 연속 3건 나오면 수집을 중단하는 조기 종료로 불필요한 조회를 줄였습니다.
- 번역 실패 기사도 원문과 메타데이터를 실패 상태로 저장해 재수집을 막았습니다.
- 실패를 timeout, 할당량 초과, 안전 정책 차단, 응답 잘림처럼 대응 방법이 다른 유형으로 분류해 저장했습니다.



3 목록 조회 성능 최적화

목록 화면에서 select(*)로 사용하지 않는 원문과 번역문까지 함께 조회해 로딩 시간이 길어졌습니다.

- 목록은 필요한 메타데이터만 명시적으로 조회하도록 바꿨습니다.
- 상세 페이지의 원문과 번역문은 토글 전환 시 지연 조회하도록 분리했습니다.
- 측정 결과 페이로드 200kB에서 16.8kB, 로딩 시간 3.14초에서 252ms로 줄였습니다.

4 음악 평론 문맥을 반영한 AI 번역 파이프라인 설계

음악 평론에는 일반 독자가 낯선 장르와 연주 기법 용어가 자주 등장해 직역만으로는 이해하기 어렵고, 긴 기사는 모델의 입출력 한도를 넘길 가능성이 있었습니다.

- 프롬프트에 어려운 평론 용어는 짧은 한국어 풀이를 함께 제공하도록 규칙을 설계했습니다.
- Gemini 모델의 실제 처리 여유를 측정해 최대 청크를 4만 자로 설정하고, 초과 시 문단 단위로 분할해 번역한 뒤 결합했습니다.
- 영문 병기는 백틱으로 감싸 식별 가능한 마크업으로 만들고, 후처리에서 두 번째 이후 구간만 삭제해 중복을 제거했습니다.
- 매체별 고정 노이즈 문구를 미리 조사해 규칙 기반 후처리로 정제했습니다.

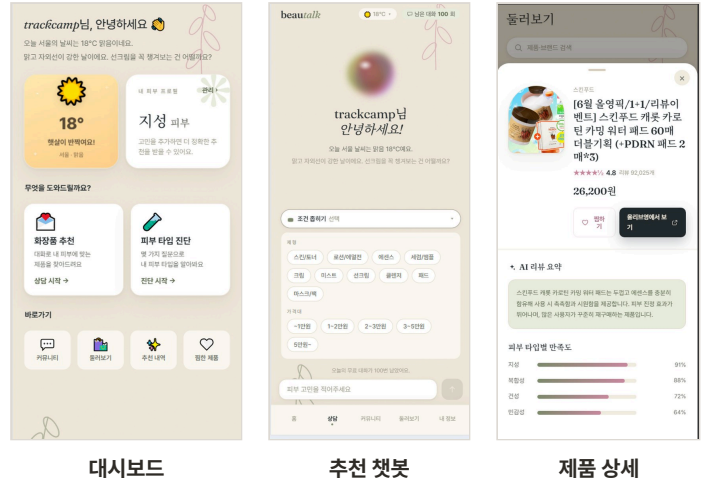
1826년, 니세포르 니엥스 Nicéphore Niépce라는 프랑스 발명가가 그의 저택 창밖으로 카메라 옵스큐라 camera obscura; 카메라의 원조가 되는 상자형 암상자를 겨냥했고, 약 6주에 걸쳐 건물과 시골 풍경을 담은 이미지를 만들어냈습니다. 기본적으로 모든 사람들은 그것이 세계에서 가장 오래 살아남은

3 beautalk : 화장품 추천 챗봇 서비스

Github

2인 / 백엔드, AI / 2026.05 ~ 06

수집한 화장품 데이터에서 사용자의 피부 고민, 가격 대, 원하는 제형에 맞는 제품을 추천하는 서비스입니다. 백엔드에서 OAuth와 JWT 인증, 찜과 추천 이력, 커뮤니티, 날씨 API 연동을 구현했습니다. 데이터와 AI 영역에서는 상품 크롤링과 정제, 리뷰 요약, 임베딩 생성, SQL 필터와 RAG를 결합한 추천 기능을 담당했습니다.



서비스 구조



추천 로직

1 추천 데이터 품질 개선

- 같은 상품이 여러 번 들어오고 있어, 판매처가 보증하는 **상품 고유키(goodsNo)**를 유니크 식별자로 두고 중복을 정리했습니다.
- 수집한 리뷰를 근거로 **제품별 AI 요약**을 생성하고, 카테고리나 브랜드, 제품명에 그 요약을 붙인 문장으로 **임베딩**을 만들었습니다.

2 SQL과 RAG를 결합한 추천 구조 설계

초기에는 "지성 피부, 2~3만원대 로션"을 요청해도 토너나 세럼처럼 다른 제형이 섞이거나 가격 조건을 벗어난 제품이 추천됐습니다. AI 요약에 가격이 담기지 않았고 임베딩에 쓴 문장에도 가격이 없어, 지켜야 하는 조건을 판별할 근거가 벡터에 없었습니다.

- 대화에서 가격, 제형, 카테고리 조건을 **규칙 기반으로 추출**하고 SQL로 조건에 맞는 상품만 먼저 선별했습니다.
- PostgreSQL의 **pgvector** 확장으로 선별된 후보 안에서 사용자 요청과 의미가 가까운 상품을 정렬했습니다.
- LLM이 고른 상품 ID를 **DB에서 다시 조회**해, 존재하지 않거나 조건을 벗어난 상품이 추천되지 않도록 검증했습니다.

3 추천 응답 시간 25초 → 3.6초 단축

추천 응답이 약 25초 걸려 사용자 경험을 해치는 문제가 있었습니다.

- LangSmith 트레이싱으로 추천 과정의 입력, 출력, 단계별 지연 시간을 기록했습니다.
- 임베딩 1초, SQL 조회 0.1초였지만 **LLM 호출이 약 24초로 전체 지연의 96%**를 차지함을 확인했습니다.
- 상품 ID를 고르는 작업에 추론 모델이 과하다고 판단해 **추론 2종과 비추론 3종을 같은 요청으로 비교**했고, 추천 품질을 유지하면서 가장 빠른 모델로 교체해 **9.1초에서 3.6초로** 줄였습니다.
- 모델명은 **환경 변수로 분리**해 운영 중에도 빠르게 교체하거나 이전 모델로 되돌릴 수 있도록 구성했습니다.

☐	☒	Name	Input	Output	Error	Start Time	Latency
☐	☒	gms_call	user: 이번엔 크림을 추천해줘	[{"category": "피부", "product": "복합성 피부..."}]		6/25/2026, 6:5...	0.95s
☐	☒	resolve_constraints	user: 이번엔 크림을 추천해줘	{ "categories": [], "forms": [...]		6/25/2026, 6:5...	0.00s
☐	☒	gms_call	user: 지금까지의 대화를 바...	[{"content": "복합성 피부..."}]		6/25/2026, 6:5...	4.45s
☐	☒	recommend_candidates	assistant: 복합성 피부에 적...	[{"Product object (519613..."}]		6/25/2026, 6:5...	1.66s
☐	☒	gms_call	user: 내 피부타입에 맞는 스...	[{"content": "복합성 피부..."}]		6/25/2026, 6:5...	0.92s
☐	☒	resolve_constraints	user: 내 피부타입에 맞는 스...	{ "categories": [], "forms": [...]		6/25/2026, 6:5...	0.00s
☐	☒	gms_call	user: 스킨/토너 제품을 1~2...	[{"content": "스킨/토너 제..."}]		6/25/2026, 6:5...	1.65s
☐	☒	resolve_constraints	user: 스킨/토너 제품을 1~2...	{ "categories": [], "forms": [...]		6/25/2026, 6:5...	0.01s
☐	☒	gms_call	user: 지금까지의 대화를 바...	[{"content": "지성 피부에 ..."}]		6/25/2026, 6:5...	4.42s
☐	☒	recommend_candidates	assistant: 알겠습니다. 지성...	[{"Product object (ac6d0..."}]		6/25/2026, 6:5...	2.11s
☐	☒	gms_call	user: 스킨/토너 제품을 1~2...	[{"content": "알겠습니다. ..."}]		6/25/2026, 6:5...	1.11s

단계마다 걸린 시간이 기록됩니다. 규칙으로 조건을 뽑는 `resolve_constraints`는 **0.00초**, LLM을 부르는 `gms_call`은 초 단위로 걸립니다.

4 날씨를 대화 맥락으로만 쓰도록 설계

- OpenWeatherMap으로 현재 날씨를 읽어 **제품을 권하는 문장이 아니라 대화의 맥락으로** 넘겼습니다. 맑으면 선크어를, 비가 오면 워터프루프를 **자연스럽게 언급**하도록 안내만 합니다.
- 같은 날씨를 매 대화마다 조회하지 않도록 **30분 캐시**를 두었습니다.
- API 키가 없거나 조회에 실패하면 **빈 문자열**을 돌려 **대화는 그대로 진행**됩니다. 부가 기능이 본체를 멈추게 하지 않도록 했습니다.

```
_WEATHER_SKIN_HINT = {
  'clear': '맑고 자외선이 강합니다. 선크어·수분 제품을 자연스럽게 언급하면 좋습니다.',
  'rain': '비가 내립니다. 워터프루프나 클렌징 제품을 언급해도 좋습니다.',
  'snow': '추운 날씨입니다. 장벽 강화·보습 크림을 자연스럽게 언급하세요.',
  'clouds': '흐리거나 대기질이 좋지 않습니다. 클렌징이나 자외선 차단을 가볍게 언급할 수 있습니다.',
}
```

날씨 상태마다 어떤 맥락을 넘길지 규칙으로 정해 두었습니다.

5 앱별 테스트를 빌드에서 자동 수행

기능을 고칠 때마다 권한 검사나 중복 요청 처리가 조용히 깨질 수 있는데, 손으로 확인하면 매번 같은 경로만 보게 됩니다.

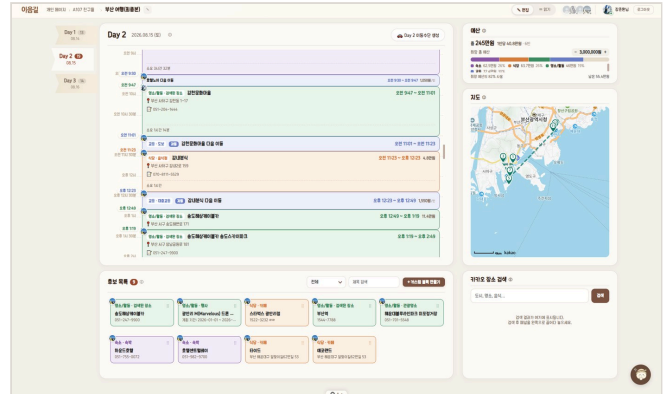
- 계정, 커뮤니티, 대화, 상품 네 개 앱에 테스트를 두고 **총 65개를 빌드 때 자동 수행**하도록 구성했습니다.
- **다른 사용자의 글을 수정하거나 삭제할 수 없는지** 검사해 권한 우회를 막았습니다.
- 좋아요를 두 번 눌러도 결과가 같은지 **멥등성**을 확인하고, 취소할 것이 없을 때는 404를 주도록 했습니다.
- 없는 리소스에는 404, 권한이 없으면 403을 주는지 **에러 코드를 구분**해 검사했습니다.

4 이음길 : 여행 일정을 함께 편집하는 협업 보드

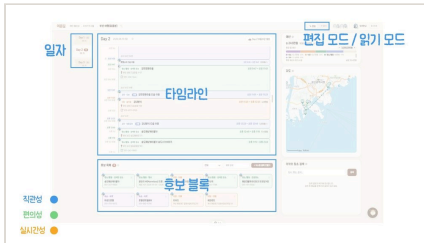
Github

6인 / 백엔드 / 2026.07 ~ 08 (4주)

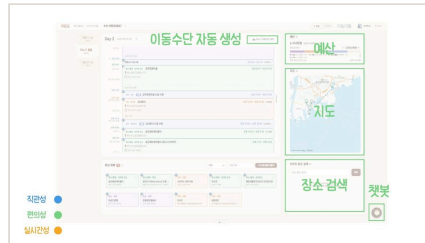
여러 명이 하나의 보드에서 일정 블록을 드래그로 옮기며 실시간으로 공동 편집하는 서비스입니다. 저는 서버측 실시간 동기화 코어를 맡았습니다. 클라이언트끼리 직접 주고받지 않고 변경을 서버로 보내면, 서버가 순서를 정해 처리한 뒤 다시 모두에게 보내는 구조로 만들었습니다. 순서 판정과 충돌 해소를 한 곳에서 하기 위해서입니다. 팀원들이 만든 대시보드 기능이 이 파이프라인 위에 쌓이는 구조라, 여기가 흔들리면 다른 작업도 함께 멈췄습니다. 동시 편집에서 생기는 충돌, 유실, 교착을 다루는 것이 제 일이었습니다.



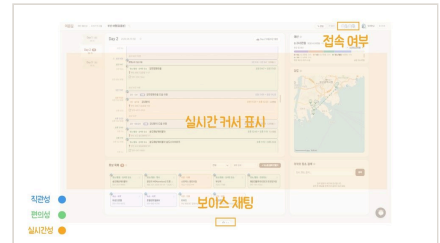
일정 보드 화면



직관성



편의성



실시간성

실시간 동기화 코어

1 두 계층의 락이 얹힌 교착 해소

op 기록 파이프라인에 프로젝트 단위 락을 두어 시퀀스 채번부터 기록까지를 직렬화했는데, 시퀀스 유실을 막으려고 그 락을 커밋 완료까지 연장하면서 JVM 락을 쥔 채 DB 커밋을 수행하는 구간이 생겼습니다. 행 락을 먼저 주고 프로젝트 락을 기다리는 트랜잭션과, 프로젝트 락을 쥔 채 같은 행 락을 기다리는 트랜잭션이 교차하면 교착이 났습니다.

- PostgreSQL의 데드락 감지기는 JVM 락을 볼 수 없습니다. DB만 보는 감지기가 풀어주지 못해 애플리케이션이 타임아웃까지 멈춰 있습니다.
- 1차로 락 획득에 상한(tryLock)을 걸어 무한 대기를 끊었습니다.
- 2차로 flush를 락 획득보다 앞세워 두 락의 획득 순서를 한 방향으로 통일하고 원인을 제거했습니다.
- 서로 다른 계층의 락을 겹치면 어느 감지기도 전체를 보지 못합니다. 락 범위는 한 계층 안에서 닫는 것을 원칙으로 삼았습니다.

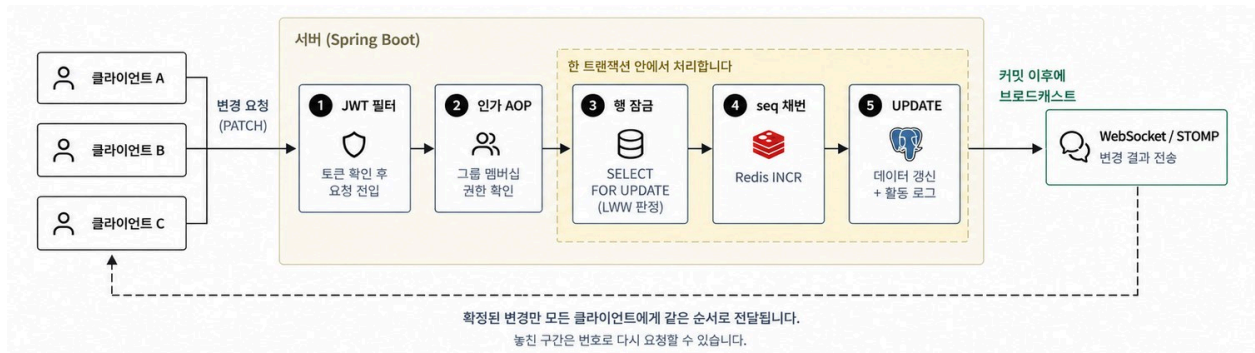
2 동시 쓰기 충돌 해소 (필드 단위 LWW)

두 사람이 같은 블록의 서로 다른 필드를 동시에 고치면, 행 단위로 덮어쓰던 구조에서는 나중 요청이 앞선 변경을 통째로 지웠습니다.

- 갱신을 필드 단위 배치로 바꾸고 필드별로 최종 승자를 판정하게 했습니다.
- 삭제는 tombstone으로 남겨 재생성과 구분했습니다.

3 순서 보장과 유실 복구

- Redis **INCR**로 시퀀스를 **채번**하고 DB 기준으로 리시드해 재시작 후에도 번호가 어긋나지 않게 했습니다.
- 브로드캐스트를 **커밋 이후**로 옮겼습니다. 커밋 전에 보내면 다른 클라이언트가 아직 없는 데이터를 먼저 받습니다.
- 클라이언트가 놓친 변경을 **번호 구간으로 재요청**하는 API를 함께 뒀습니다.



4 쿼리 수 측정 후 감축

- 대시보드 스냅샷 API를 **쿼리 4회 고정**으로 설계했습니다. 블록, 멤버, 활동로그를 따로 조회하면 참여자 수에 따라 늘어납니다.
- 멤버십 검증에서 복합키 엔티티의 exists 쿼리가 두 테이블을 조인해 전 컬럼을 읽고 있어 **JPQL count로 교체해 조인을 제거**했습니다.
- 그룹 목록 조회 한 번에 나가던 **쿼리 21회를 Fetch Join으로 1~2회로 줄였습니다**.

5 실시간 채널의 인증과 인가 점검

- CONNECT 시 JWT 인증, SUBSCRIBE와 SEND 목적지 인가, 세션 레지스트리, 하트비트를 구성했습니다.
- 이후 점검에서 **세션 인증에 만료가 없어 탈퇴 후에도 실시간 수신**이 가능한 문제를 찾아 고쳤습니다.
- 보이스 시그널 대상을 검증하지 않아 임의의 멤버에게 오퍼를 보낼 수 있던 문제도 함께 고쳤습니다.

6 반복 검증의 AOP 분리와 회귀 테스트

- **@GroupMember 애노테이션 하나로** groupId, projectId, blockId 어느 파라미터에서든 그룹을 역추적해 검증합니다.
- 404와 403의 순서를 지켜 **존재 여부와 권한 여부를 구분**했습니다.
- 회귀 테스트 **약 1,200행**으로 LWW, 동시성, tombstone, 커밋 후 브로드캐스트, 잠금 경합과 만료, 인가, 시퀀스 리시드를 덮었습니다. 위 결함들이 다시 생기면 테스트가 먼저 잡습니다.

5 Finch : 가상 계좌 기반 모의투자 서비스

6인 / 인프라와 운영 환경 / 2026.08 ~ 진행 중

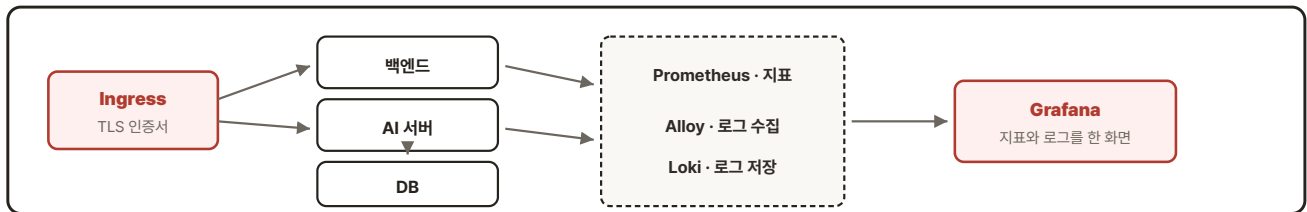
공시와 뉴스에서 내 종목에 필요한 정보만 골라내는 모의투자 서비스입니다. Docker Compose로는 무중단 교체와 준비되지 않은 컨테이너 제외를 사람이 챙겨야 해 서버 한 대 규모에 맞는 경량 쿠버네티스 배포판 k3s로 옮겼습니다. 오래 걸린 것은 설치가 아니라 이미 붙어 있던 것을 빠짐없이 찾는 일이었습니다.



서비스 화면

k3s 단일 노드

배포 구성은 Helm 차트로 정의



배포 전 CI 검사

환경변수 목록과 실제 주입 항목 대조

매니페스트 유효성

배포 후 스모크 테스트

옮기고 나서야 드러난 것들

1 Compose가 가려 주던 작은 설정

같은 이미지를 Helm으로 배포했더니 동작이 달라졌습니다. Compose의 **environment**가 값을 대신 채워 주고 있어 비어 있던 설정이 그동안 드러나지 않았습니다.

- 이전 환경이 조용히 메워 주던 것은 옮기기 전에는 목록에 잡히지 않습니다.
- 필요한 환경변수와 배포에 주입하는 항목을 CI에서 대조하게 해 같은 누락을 배포 전에 잡습니다.

2 검사 대상이 실제와 달랐던 일

API 키 검사 스크립트는 통과하는데 데이터가 쌓이지 않았습니다. 스크립트가 본 것은 빌드 환경의 키였고, 애플리케이션이 읽는 컨테이너의 키는 비어 있었습니다.

- 점검 대상을 실행 환경이 실제로 읽는 곳으로 바꾸고, 접근 실패와 키 없음을 구분했습니다.

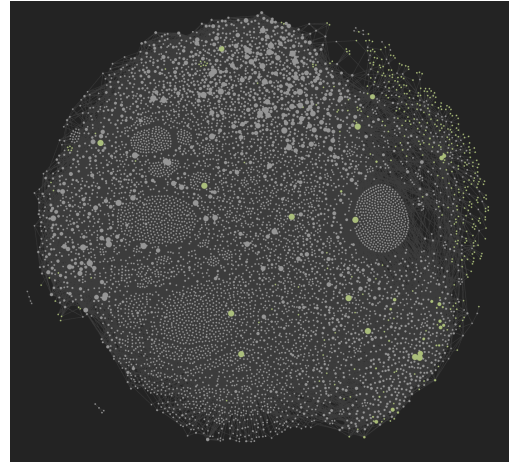
3 관측 스택은 앱 코드를 건드리지 않는 쪽으로

- 앱에서 로그를 적재하면 팀원 모두가 그 코드를 넣어야 합니다. Alloy가 컨테이너 로그를 긁어 Loki로 보내는 구조를 골라 앱 파트의 적재 코드는 0줄입니다.
- 흠이 최대 세 단계라 분산 추적 없이 엔드포인트별 응답 시간 분포와 요청 식별자 전파로 병목을 좁혔습니다.

6 세컨드 브레인 : 음악 지식 베이스

1인 / 2026.07 ~ 진행 중

음악 칼럼을 읽고 용어를 정리하고 인사이트를 뽑아 사고를 넓히려고 만들었습니다. 그 전에도 자료를 찾아 노선에 모아뒀지만, 필요할 때마다 다시 찾아 읽어야 했고 예전에 잘못 적어둔 것을 발견하기 어려웠습니다. 2026년 7월부터 AI를 적극 활용하되 확실한 정보만 적재하고, 충돌하는 내용은 검증하며, 기존 자료의 오류까지 검출하는 구조로 다시 만들었습니다. 칼럼 하나를 읽으면 명제 단위로 쪼개 아티스트, 레이블, 장르에 걸어둡니다. 지금까지 노트 7,226개를 쌓았고 그 사이에 연결 35,917개가 생겼습니다.



노트 사이 연결을 그린 그래프

규칙과 판정 기준

1 규칙을 먼저 정하고 AI는 그 안에서만

AI로 노트를 대량 생성하면 빠르지만, 근거 없는 서술이 섞여도 그대로 쌓입니다. 잘못된 사실은 나중에 다른 노트와 충돌해야 겨우 드러납니다.

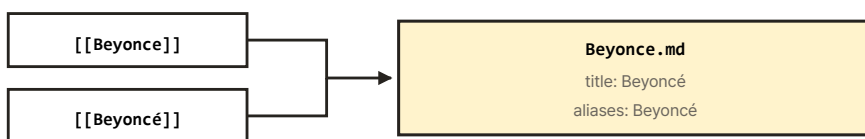
- 볼트 정책 문서에 "문서가 존재하고 대상이 정확히 특정된 경우에만 작성한다. 없거나 불명확하면 건너뛰거나 보류한다"를 최우선 기준으로 뒀습니다.
- "근거 없는 평가나 출처 밖의 추측으로 내용을 대체하지 않는다"를 함께 명시하고, 원문 URL을 노트마다 남기게 했습니다.
- 노트 하나를 완료로 표시하기까지 순서를 정해 뒀습니다. 대상이 맞는지 확인하고, 원문을 다 읽고, 사실의 누락과 왜곡부터 검토한 뒤, 메타데이터와 링크를 걸고, 기존 앨범과 리뷰가 지워지지 않았는지 확인합니다. 여덟 단계를 모두 통과한 뒤에만 완료로 표시합니다.

2 동명이인 판정을 사람에게 넘기는 기준

아티스트 이름은 겹칩니다. 자동으로 연결하면 조용히 틀린 관계가 쌓이고, 사람이 전부 확인하면 규모를 감당할 수 없습니다.

- 정책에 "동음이의어나 국적이 불명확한 대상은 자동 링크나 스텝을 만들지 않고 검토 목록에 기록한다"를 규칙으로 뒀습니다.
- 판정 스크립트가 문서 존재 여부, 동음이의 문서인지, 음악가가 아닌지, 한국 아티스트가 영문명으로 들어와 있는지를 확인해 다섯 가지로 분류합니다.
- 스크립트는 판정 표만 출력하고 노트는 건드리지 않습니다. 확실한 것과 애매한 것을 나눠, 애매한 것만 사람이 봅니다.

규칙 하나를 예로 들면, 파일명



둘 다 같은 노트로 연결

폴더와 파일명은 ASCII로,
원표기는 제목과 별칭에 둡니다

그대로 두면 콘솔에서 깨집니다
Foushéé → Foushe첵